

Deployment Profiles: Evaluated Configurations from Part 2

Deployment Profiles: Evaluated Configurations from Part 2 I

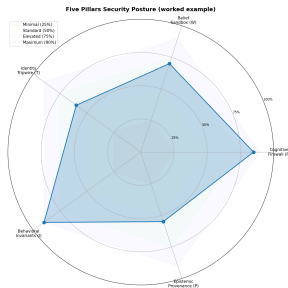


Figure 1: Five-Pillar operator posture assessment radar (Cognitive Firewall, Belief Sandbox, Identity Tripwire, Behavioral Invariants, Provenance), color-coded against readiness thresholds.

Deployment Profiles: Evaluated Configurations from Part 2 II

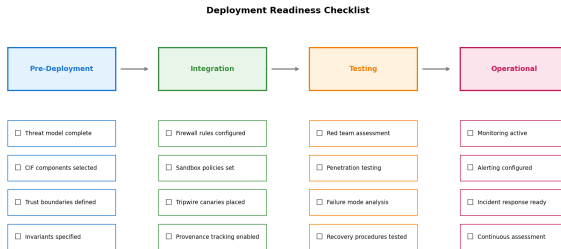


Figure 2: Pre-deployment → Integration → Testing → Operational checklist flowchart mapping CIF enforcement points to deployment phases.

Deployment Profiles: Evaluated Configurations from Part 2 I

In Part 2, we evaluated specific configurations of the Cognitive Integrity Framework to understand how different tuning parameters affected security and performance outcomes. The following profiles are derived directly from the **Parameter Sensitivity Analysis** (Part 2) and **Architecture-Specific Results** (Part 2).

Profile A: The “Internal Tool” Baseline (Low Latency)

Profile A: The “Internal Tool” Baseline (Low Latency) I

This profile corresponds to the “High Usability” configuration tested in the sensitivity analysis ($\delta = 0.95$). It is designed for low-risk, human-in-the-loop environments.

Configuration Parameters:

Profile A: The “Internal Tool” Baseline (Low Latency) I

- ▶ **Trust Decay (δ):** 0.95. Maintained >50% trust retention even after 13 delegation hops.
- ▶ **Firewall Sensitivity:** Relaxed (reject threshold $\tau_1 = 0.9$).
- ▶ **Consensus:** Simple Majority.

Profile A: The “Internal Tool” Baseline (Low Latency) I

Modelled performance (Part 2, parametric parameter-sensitivity analysis — simulation output under calibrated conditions, not a measurement of a running deployment):

Profile A: The “Internal Tool” Baseline (Low Latency) I

- ▶ **Latency Overhead:** Minimal (~15% baseline).
- ▶ **Detection Rate:** **87%** (vs 94% optimal).
- ▶ **Trade-off:** The high trust decay allows for fluid, deep delegation chains but increases vulnerability to subtle trust laundering (Ω_4).

Profile B: The “Customer Facing” Baseline (Balanced)

Profile B: The “Customer Facing” Baseline (Balanced) I

This profile corresponds to the parametrically optimal configuration identified in Part 2 (§{S08}, “Empirically Optimal Configuration (Parametric)”) (Architecture-Specific Results), which balances security guarantees with operational overhead.

Configuration Parameters:

Profile B: The “Customer Facing” Baseline (Balanced) I

- ▶ **Trust Decay (δ):** 0.80. At this setting, trust degrades to $<50\%$ after 4 hops, strictly bounding the “radius of effective delegation.”
- ▶ **Firewall Sensitivity:** Balanced (reject threshold $\tau_1 = 0.7$; quarantine threshold $\tau_2 = 0.5$).
- ▶ **Consensus:** Variable (Architecture Dependent).

Profile B: The “Customer Facing” Baseline (Balanced) I

Modelled performance (Part 2, parametric parameter-sensitivity analysis — simulation output under calibrated conditions, not a measurement of a running deployment):

Profile B: The “Customer Facing” Baseline (Balanced) I

- ▶ **Latency Overhead:** Reduced detection latency (~8.5s for drift detection).
- ▶ **Detection Rate: 94%.**
- ▶ **Resilience:** Maximizes the F1 score, providing the best empirically observed trade-off between False Positives (0.06) and True Positives.

Profile C: The “Autonomous Operator” Baseline (High Assurance)

Profile C: The “Autonomous Operator” Baseline (High Assurance) I

This profile corresponds to the “Byzantine-Heavy” configuration tested in Part 2 (Byzantine Consensus Analysis). It is required for high-stakes, unsupervised environments.

Configuration Parameters:

Profile C: The “Autonomous Operator” Baseline (High Assurance) I

- ▶ **Trust Decay (δ):** 0.60. Aggressive decay. Trust halves every ~ 1.36 hops, enforcing a strictly flat command structure.
- ▶ **Firewall Sensitivity:** Strict (reject threshold $\tau_1 = 0.4$).
- ▶ **Consensus:** Byzantine Fault Tolerance ($n \geq 3f + 1$).

Profile C: The “Autonomous Operator” Baseline (High Assurance) I

Modelled performance (Part 2, parametric parameter-sensitivity analysis — simulation output under calibrated conditions, not a measurement of a running deployment):

Profile C: The “Autonomous Operator” Baseline (High Assurance) I

- ▶ **Latency Overhead:** Significant (+35-48% depending on corroboration count).
- ▶ **Detection Rate:** **95%** (general) to **98%** (LangGraph-specific).
- ▶ **Resilience:** Specifically tunes parameters to resist coordination attacks (Ω_4), accepting higher latency for a near-zero semantic drift rate.

Architecture-Specific Observations

Architecture-Specific Observations I

Beyond the parameter profiles, Part 2's architecture adapters revealed specific interactions between the defense framework and the underlying agent topology.

Observation: LangGraph architectures achieved the highest overall detection rates (98%) in our tests (Part 2, Architecture-Specific Results). **Mechanism:** The explicit definition of state transitions allowed for rigorous **Invariant Checking**. Invalid state transitions were detected deterministically by the framework.

Observation: trust-exploitation attacks are the category the parametric simulation covers most completely: Claude Code, CrewAI and LangGraph all reach 100% and AutoGPT 96%, on 200 attacks each (Part 2, Architecture-Specific Results). No architecture is distinctively better here, and the figures are design-level rather than measured on a deployment. **Mechanism:** The framework's role definitions acted as implicit **Identity Tripwires**. When an agent attempted to act outside its defined role, the behavior was flagged as a role violation.

Minimal Viable Implementation

Minimal Viable Implementation I

We also evaluated a “Minimal Viable Implementation” (MVI) to determine the baseline efficacy of the framework’s core components.

The Setup:

Minimal Viable Implementation I

1. **Trust Decay:** $\delta = 0.80$ (The optimal balance point).
2. **Cognitive Firewall:** Ingress only.
3. **Tripwires:** One per agent.

Minimal Viable Implementation I

Result: Even this minimal setup shifted the success rate against low-effort attacks from 100% (Baseline) to $<5\%$, providing a critical first line of defense.

Alignment with Emerging Standards

Alignment with Emerging Standards I

Practitioners deploying cognitive security must increasingly demonstrate compliance with industry and government standards. CIF's defense mechanisms map directly to two major 2025–2026 standardization efforts.

OWASP Top 10 for Agentic Applications (2026) I

The OWASP Agentic Top 10 identifies ten risks (ASI01–ASI10) specific to autonomous multiagent deployments. CIF addresses the majority through its layered defense architecture:

OWASP Top 10 for Agentic Applications (2026) I

OWASP Risk	CIF Defense	Profile Coverage
ASI01: Agent Goal Hijack	Cognitive Firewall + Tripwires	All profiles
ASI02: Tool Misuse/Exploitation	Belief Sandbox (tool output isolation)	Profiles B, C
ASI03: Identity/Privilege Abuse	Trust Calculus (δ^d decay)	All profiles
ASI06: Memory/Context Poisoning	Tripwire monitoring + Drift detection	Profiles B, C
ASI07: Insecure Inter-Agent Comm.	Provenance attestation	Profiles B, C
ASI08: Cascading Failures	Byzantine Consensus	Profile C
ASI10: Rogue Agents	Full CIF stack	Profile C

NIST Zero Trust Architecture for AI Agents I

NIST's extension of SP 800-207 to AI agents establishes “never trust, always verify” principles. CIF operationalizes zero trust for cognitive interactions:

NIST Zero Trust Architecture for AI Agents I

- ▶ **Continuous verification:** Every inter-agent message is evaluated by the Cognitive Firewall
- ▶ **Micro-segmentation:** Beliefs from external sources are sandboxed before integration
- ▶ **Least privilege:** Trust scores decay exponentially with delegation depth (δ^d)
- ▶ **Continuous authentication:** Provenance attestation provides cryptographic message origin tracking

NIST Zero Trust Architecture for AI Agents I

Profile A (Internal Tool) provides partial NIST alignment. Profile B (Customer Facing) achieves substantial compliance. Profile C (Autonomous Operator) is the profile that maps most completely to the controls cited in the OWASP and NIST frameworks above; treat any mapping as design intent, not certification.

Composer Data Bundle

Part 2 does not ship an interactive planning application. What it ships is the data such a tool would consume: `scripts/generate_composer_data.py` writes `output/data/composer_data.json`, which bundles the eight defense modules' descriptions, the attack families each is meant to handle, and the measured detection rate and latency for each, hydrated from `output/data/module_capability_matrix.json` (Part 2, §{S12}).

An operator planning a deployment can therefore read the per-module capability directly rather than compose it in a browser, and the numbers are the measured ones rather than the design-level ceiling: on the full 1,475-item corpus the invariants checker reaches 83.3% alone and no other module exceeds 10%. Any composition estimate built on top of that bundle should carry the caveat the ablation establishes — the modules are not independent, and the series-composition rule over-predicts the union rate.